System Request – Smart Queue System

Project Name:
Queue Flow – Smart Queue Management System

Project Sponsor:
Dean of Student Affairs

Name:
Pavly Gamil (MANAGER)
Thomas Shenouda

Department: Student Affairs
Phone: 010XXXXXXX
Email: studentaffairs@college.com

Purpose of the Project

The purpose of the StudentQueue project is to develop a Smart Queue Management System that helps organize and manage student queues in universities and educational institutions. The system aims to reduce waiting times, prevent overcrowding, and improve the overall student experience when accessing university services, such as student affairs, registration, and payment offices.

The project also aims to make the queuing process more efficient and organized by allowing students to obtain queue numbers and monitor their turn easily. This will help university staff manage students more effectively, save time, and improve the quality of services provided.

Ultimately, StudentQueue aims to create a more convenient, organized, and efficient environment for both students and university employees.

Business Case: Smart Queue Management System (Student Queue)

Executive Summary

In academic environments, student service centers, administrative offices, registration hubs, and financial affairs departments experience massive foot traffic, especially during peak periods such as enrollment, exams, and document issuance. The current manual or traditional paper-based queue systems lead to severe campus overcrowding, long waiting times, administrative inefficiencies, and high levels of student frustration.

The **Smart Queue Management System (Student Queue)** addresses these operational bottlenecks by providing a digital, real-time queue platform that enables students to join, monitor, and manage their spot in line directly from their smartphones using QR code technology.

Problem Statement

● **Severe Campus Overcrowding:** Students physically gather and wait for hours in tight corridors and waiting areas, creating an uncomfortable and inefficient environment.

● **Lack of Transparency & Time Loss:** Students cannot estimate their actual waiting time, leading to wasted academic hours that could be spent in lectures or libraries.

● **High Operational Stress on University Staff:** Administrative staff face high pressure dealing with unsorted, frustrated crowds without clear metrics on student volume and service demands.

● **Data Blindness for University Administration:** Absence of operational metrics makes it impossible to evaluate staff performance, measure average service duration, or plan resource allocation during peak academic cycles.

Proposed Solution

A comprehensive, web-based digital queue solution tailored specifically for university campuses:

- **Student Interface:** Allows students to scan a QR code or access a web portal to generate a virtual ticket, check their real-time queue position, and receive notifications when their turn approaches.

- **Staff/Window Dashboard:** Equips university personnel with a streamlined dashboard to call the next student, mark attendance, transfer tickets between departments, or handle priority cases.

- **Public Display Board:** Shows live ticket numbers and designated service counters on hall screens with audio prompts.

- **Analytics Engine:** Generates data-driven insights on student flow, average waiting times, and service completion rates.

Expected Benefits & Value Proposition

- **Reduction in On-Site Waiting Crowds:** Students can wait anywhere on campus (cafeteria, library, outdoors) until their turn draws near.

- **Optimized Academic Time:** Minimizes missed lectures and study hours by eliminating static line waiting.

- **Enhanced Operational Efficiency:** Streamlines administrative workflows, allowing staff to serve more students per hour with reduced stress.

- **Data-Driven Campus Governance:** University administrators gain actionable insights into peak activity hours and department bottlenecks, enabling better staffing and scheduling decisions.

Project Scope Statement

Queue Flow – Smart Queue Management System

1. Product Scope Description

The Queue Flow project is a web-based Smart Queue Management System designed to organize and manage student queues in universities and educational institutions.

The system will allow students to join a virtual queue using a QR code or web/mobile interface instead of physically waiting in crowded service areas. Students will be able to obtain a virtual ticket, monitor their current queue position, view estimated waiting times, and receive notifications when their turn is approaching.

The system will also provide university staff with an administrative dashboard to manage queues, service counters, student tickets, and priority cases. Public display screens will show currently served ticket numbers and assigned service counters in real time.

The system will include an analytics component that provides basic operational information such as average waiting time, service duration, queue volume, and peak activity periods.

The main product components are:

- Student Web/Mobile Interface.
- QR Code Ticket Generation.
- Real-Time Queue Tracking.
- Staff and Administrative Dashboard.
- Service Counter Management.
- Public Display Board.
- Notifications and Alerts.
- Queue Analytics and Reports.
- Database and Backend Services.

- Real-Time Communication using WebSockets.

2. Product Acceptance Criteria

- Students can generate a unique queue ticket.
- Students can track their position and estimated waiting time.
- Staff can call, serve, skip, and manage tickets.
- Public screens display current queue information.
- Queue updates are synchronized in real time.
- Authorized administrators can view basic queue statistics.
- The system passes functional and basic performance testing.

3. Project Deliverables

- Requirements / SRS Document.
- ERD and UML Diagrams.
- UI/UX Designs.
- Student Interface.
- Staff/Admin Dashboard.
- Public Display Interface.
- Backend APIs and Database.
- Real-Time Queue Management.
- QR Code and Notification System.
- Testing Results.
- Deployed System and Final Documentation.

4. Project Exclusions

- Online payment processing.
- Replacing the university's existing systems.
- Advanced AI/ML features.
- Facial or biometric recognition.
- Advanced predictive analytics.
- Large-scale hardware and network infrastructure.
- Features outside the approved project scope.
- ## 5. Project Constraints
- The Queue Flow project will operate under the following constraints:
- **Time Constraint**
- The project is planned to be completed within the defined project schedule. Delays in requirements analysis, development, testing, or integration may affect the final delivery date.
- **Budget Constraint**

- The project must remain within the approved budget for cloud hosting, domain and SSL, real-time communication services, notification services, software tools, and testing hardware.
- **Scope Constraint**
- The project must focus on the core queue management functions. Additional advanced features should not be added without formal approval.
- **Human Resource Constraint**
- The project depends on a limited development team. Team members must distribute responsibilities according to their skills and availability.
- **Technical Constraint**
- The system depends on stable internet connectivity, compatible devices, database availability, and reliable real-time communication services.
- **Security and Privacy Constraint**
- The system must protect student information and restrict administrative functions to authorized users.
- **User Adoption Constraint**
- Students and university employees may have different levels of familiarity with digital queue systems. Therefore, the system must provide a simple and user-friendly interface.

6. Project Assumptions

- Students have smartphones and internet access.
- The university provides suitable locations for QR codes and displays.
- Staff will receive basic system training.
- Required hosting and database services will be available.
- The university approves the use of the digital queue system.

WBS Approach

The **Queue Flow – Smart Queue Management System** WBS will be developed using the **Top-Down Approach**, starting from the general project scope and decomposing it into more specific deliverables and work packages.

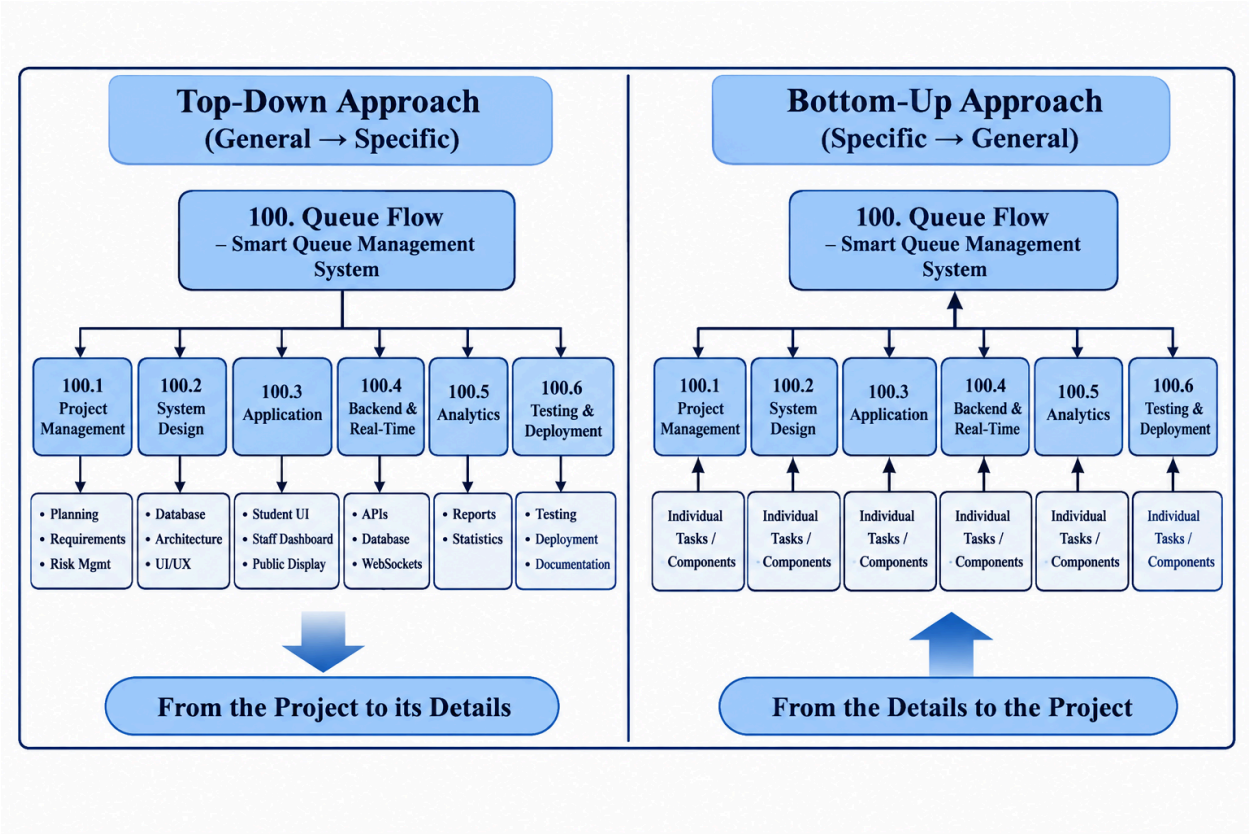
Top-Down Approach (General → Specific)

100. Queue Flow – Smart Queue Management System

- **100.1 Project Management**
    - 100.1.1 Planning
    - 100.1.2 Requirements
    - 100.1.3 Risk Management
- **100.2 System Design**

    - 100.2.1 Database Design
    - 100.2.2 System Architecture
    - 100.2.3 UI/UX Design
- **100.3 Application**
    - 100.3.1 Student Interface
    - 100.3.2 Staff Dashboard
    - 100.3.3 Public Display
- **100.4 Backend & Real-Time**
    - 100.4.1 APIs
    - 100.4.2 Database
    - 100.4.3 WebSockets
- **100.5 Analytics**
    - 100.5.1 Reports
    - 100.5.2 Statistics
- **100.6 Testing & Deployment**
    - 100.6.1 Testing
    - 100.6.2 Deployment
    - 100.6.3 Documentation

This approach starts with the complete Queue Flow project and breaks it down into smaller and more manageable components until reaching the specific work packages.



Platforms

The StudentQueue project will be designed to operate across multiple platforms to make the system accessible and easy to use for students and university staff.

1. Web Platform:
The web platform will allow students to access the system through a web browser to obtain queue numbers, check their current position in the queue, and view estimated waiting times. University staff will use the platform to manage queues and organize student services.

2. Mobile Platform:
The mobile platform will allow students to access queue information using their smartphones. Students will be able to check their queue status and receive notifications when their turn is approaching, helping them save time and avoid unnecessary waiting.

3. Administrative Dashboard:
The administrative dashboard will provide university employees with tools to manage student queues, monitor waiting times, control service counters, and view basic statistics about the system's performance.

4. Display Screens:
Digital screens placed in university service areas will display queue numbers, currently served students, and the next numbers to be served. This will help students follow the queue easily and reduce confusion and overcrowding.

These platforms will work together to provide a more organized, efficient, and convenient queue management experience for students and university staff.

Project Milestones (Student Queue Management System)

Milestone 1: Planning, Requirements & System Architecture (Weeks 1 – 2)

- **Requirement Gathering & SRS Document:** Defining student workflows, administrative needs, and peak-hour scenarios across university departments.

- **System & Database Design:** Creating Entity-Relationship Diagrams (ERD), UML Use-Case diagrams, and defining data structures for students, counters, services, and live tickets.

- **UI/UX Wireframing:** Designing user-friendly prototypes for the Student Mobile Portal, Staff Dashboard, and Public Display Interface.

- **Milestone Deliverables:** Approved SRS document, ERD/UML diagrams, and Figma UI prototypes.

Milestone 2: Core Development & Real-Time Integration (Weeks 3 – 5)

- **Backend & API Development:** Building robust RESTful APIs for student authentication, ticket generation, and queue management logic.

- **Real-Time Communication Setup:** Implementing WebSockets (Socket.io / SignalR) for live position updates, instant notifications, and public screen syncing.

- **Frontend Implementation:**

    - **Student Portal:** QR Code scanner/generator, live queue position tracker, and sound/push alerts.

    - **Staff & Admin Dashboard:** Counter interface to call the next student, mark attendance, transfer tickets, and view daily queue statistics.

    - **Public Display Interface:** Real-time board showing active ticket numbers and counter assignments with audio cues.

- **Milestone Deliverables:** Fully functional core application integrated with real-time backend updates.

Milestone 3: Testing, Analytics & Final Presentation (Weeks 6 – 7)

- **Testing & Optimization:** Conducting system load testing (simulating peak student registration traffic), fixing bugs, and ensuring low-latency WebSocket connections.

- **Analytics Engine:** Implementing the reporting module for administration to track average wait times, service durations, and peak hours.

- **Deployment:** Hosting the application on a cloud server (e.g., Render / Vercel / AWS) and setting up campus QR codes.

- **Documentation & Final Presentation:** Completing the final project documentation, user manual, and preparing the presentation/demonstration for the evaluation committee.

- **Milestone Deliverables:** Deployed working system, live demonstration, and final project documentation.

Project Resources

The StudentQueue project requires several resources to ensure proper planning, organization, and successful implementation. These resources include human resources, hardware, software, and financial resources.

1. Human Resources

The project requires a small team responsible for planning and managing the project. Team members may include a Project Manager, Business Analyst, UI/UX Designer, Software Developer, and Tester. Each member will have specific responsibilities to ensure that project activities are completed according to the planned schedule.

2. Hardware Resources

The project may require computers or laptops for the development team, smartphones for testing the student interface, and digital display screens to show queue numbers and service updates in university facilities. Internet connectivity and networking equipment may also be required.

3. Software Resources

The project may use project management tools such as Trello or Microsoft Project to organize tasks and schedules. Design tools such as Figma may be used to create interface prototypes, while development and database tools may be selected during the technical planning stage.

4. Financial Resources

A budget may be needed for software subscriptions, hosting services, internet access, testing equipment, and digital display screens. The project team should estimate these costs and monitor spending to keep the project within its planned budget.

5. Time Resources

Time is an essential resource for the StudentQueue project. The team must allocate sufficient time for project planning, requirements gathering, interface design, documentation, risk assessment, and project review. A project schedule should be prepared to ensure that all deliverables are completed on time.

Conclusion

Effective resource management will help the StudentQueue project team use available resources efficiently, control costs, avoid unnecessary delays, and achieve the project's objectives within the planned time and budget.

Project Communication Plan

Queue Flow – Project Communication Plan

The Project Communication Plan defines how project information will be shared between the project team and stakeholders. It identifies the information to be communicated, the target audience, timing, communication method, and responsible person.

What Information	Target Audience	When?	Method	Provider
Project Progress Report	Project Manager & Team	Weekly	Meeting / Email	Project Manager
Team Status Report	Project Manager	Weekly	Email / Meeting	Team Members
Issues & Risks Report	Project Manager & Team	When needed	Meeting / Report	Team Members
Requirements Updates	Project Manager & Team	During Requirements Phase	Meeting / Document	Team Members
Design Updates	Project Manager & Team	During Design Phase	Meeting / Presentation	Thomas Shenouda
Development Progress	Project Manager & Team	Weekly	Meeting / Report	Thomas Shenouda

Milestone Report	Project Supervisor	At each milestone	Written Report	Project Manager
Change Requests	Project Manager & Team	When needed	Email / Meeting	Team Members
Testing Results	Project Manager & Team	During Testing	Report / Meeting	Team Members
User Feedback	Project Team	During Testing	Survey / Interview	Project Manager
Final Project Review	Project Supervisor	At project completion	Final Report / Meeting	Project Manager

Why is it important?

The communication plan keeps the Queue Flow project team and stakeholders informed about project progress, requirements, problems, risks, and important decisions. It also reduces misunderstandings and ensures that the right information reaches the right person at the right time.

---

2. Responsibility Matrix

The Responsibility Matrix assigns project tasks to team members and identifies who is responsible for completing each task and who provides support. This helps prevent overlapping responsibilities and ensures that every major project activity has an assigned owner.

Symbols Used

- **R – Responsible:** The team member responsible for completing the task.
- **S – Supports / Assists:** The team member who supports the responsible member.

Queue Flow – Responsibility Matrix

Project Task	Pavly Gamil	Thomas Shenouda
Project Planning	**R**	S
Requirements Gathering	**R**	S
System Analysis	S	**R**
System Architecture	**R**	S
UI/UX Design	S	**R**
Database Design	**R**	S
Student Interface	S	**R**
Staff/Admin Dashboard	**R**	S
Backend & API Planning	**R**	S
Real-Time Communication	S	**R**
Public Display Interface	S	**R**
Testing Planning	**R**	S
System Testing	**R**	S
Risk Management	**R**	S
Documentation	S	**R**
Final Project Review	**R**	S

Final Presentation	**R**	S

Why is it important?

The Responsibility Matrix makes each team member's role clear, improves coordination, and makes it easier for the Project Manager to follow up on tasks and project progress.

---

3. Difference Between the Two Concepts

Concept	Main Question	Purpose
Communication Plan	Who needs what information, when, and how?	Organizes the flow of project information.
Responsibility Matrix	Who is responsible for each task?	Organizes task ownership and assistance.

Budget (Student Queue Management System)

While the system can leverage student-tier plans to reduce costs, a realistic commercial/production-grade budget breakdown for deploying the **Student Queue Management System** on campus is outlined below:

Budget Breakdown:

Category	Description / Tool	Estimated Cost
Cloud Infrastructure & Hosting	Production Server & Database (AWS / DigitalOcean / Heroku / Supabase)	**$30.00 – $60.00** / month

Domain Name & SSL Certificate	Custom Campus Domain (.edu / .com) + SSL Security	**$15.00 – $25.00** / year
Real-time Engine & Notification Gateway	Managed WebSockets (Pusher/Ably) + SMS/WhatsApp API Credits (Twilio)	**$20.00 – $50.00**
Development & Design Tools	Software Licenses, Prototyping (Figma Pro, Postman Team, GitHub Team)	**$0.00 – $30.00**
Hardware Testing & Deployment	QR Code Stands, Test Tablet / Dedicated Display Screen	**$50.00 – $100.00**
Total Estimated Budget	**Production-Ready Deployment**	**$165.00 – $265.00**

Budget Justification:

- **Reliable Server Infrastructure:** Upgrading to paid cloud hosting ensures low latency and high availability when hundreds of students connect simultaneously during peak registration hours.

- **Automated Student Notifications:** Allocated funds cover SMS or WhatsApp API credits to send automated alerts directly to students when their turn approaches.

- **Hardware Setup:** Covers physical QR code banners/stands across campus corridors and test displays for counter screens.

Project Constraints

The StudentQueue project may face several constraints that could affect its planning, schedule, budget, and overall success. These constraints must be identified and managed carefully to ensure that the project achieves its objectives within the available resources.

1. Time Constraint

The project must be completed within a predefined schedule. The team needs to allocate sufficient time for requirements analysis, project planning, interface design, documentation, and reviewing the final deliverables. Any delays in completing these activities may affect the overall project deadline.

2. Budget Constraint

The project must operate within an estimated budget. Potential expenses may include software subscriptions, hosting services, internet access, and digital display screens. The team should prioritize essential features and avoid unnecessary expenses to keep the project financially manageable.

3. Scope Constraint

The project scope must be clearly defined to prevent unnecessary changes or additional features during the project. The main scope includes queue number generation, real-time queue status updates, estimated waiting times, and queue management for university services. Advanced features may be considered for future development.

4. Human Resources Constraint

The project team consists of a limited number of members, so tasks must be distributed according to each member's skills and availability. Limited experience or overlapping responsibilities may affect productivity and cause delays.

5. Technical Constraint

The system may depend on stable internet connectivity, compatible devices, and reliable database services. Technical limitations could affect the accuracy of queue updates and the availability of the system. The project plan should consider these dependencies when defining the system requirements.

6. Security and Privacy Constraint

The system must protect students' personal information and prevent unauthorized access to administrative functions. Only the information necessary for managing queues should be collected, and appropriate access permissions should be considered during the planning stage.

7. User Adoption Constraint

Some students or university employees may be unfamiliar with digital queue management systems or may prefer traditional queues. Therefore, the project should consider a simple and user-friendly interface, clear instructions, and an easy transition from the existing process.

Conclusion

Managing these constraints is essential for the success of the StudentQueue project. By defining the project scope, estimating the budget, preparing a realistic schedule, and identifying technical and organizational limitations, the team can reduce potential problems and improve the chances of delivering the project successfully.

High-Level Risks (Student Queue Management System)

Below are the key risks identified for the **Student Queue Management System**, along with their impact level and mitigation strategies:

1. High Concurrency & Network Congestion During Peak Hours

- **Risk Description:** During peak registration or document collection days, hundreds of students may attempt to join the queue or scan QR codes simultaneously, causing server lag or WebSocket disconnections.
- **Impact: High** (System slowdown or temporary outage).
- **Mitigation Strategy:**
    - Use lightweight, scalable real-time protocols (WebSockets with Socket.io / SignalR).
    - Implement connection pooling and atomic database operations to process queue requests sequentially without duplicate ticket allocation.
    -

2. "No-Show" Students & Queue Stagnation

- **Risk Description:** Students may generate a ticket and leave the area, resulting in staff calling numbers of absent students and wasting administrative time.
- **Impact: Medium** (Reduced staff efficiency and delayed queue progression).
- **Mitigation Strategy:**
    - Set a auto-skip timer (e.g., 2–3 minutes) for unattended calls.
    - Send push/browser alerts when a student is 3 to 5 spots away from being served.

    - Allow staff to mark a ticket as "Absent/Skipped" with an option for the student to rejoin at a penalized position.

3. Student Network Connectivity Issues (Wi-Fi / Cellular Data)

- **Risk Description:** Weak campus Wi-Fi or cellular signal might prevent students from receiving live updates on their ticket status.
- **Impact: Medium** (Missed turns or inaccurate wait-time tracking).
- **Mitigation Strategy:**
    - Display active numbers on prominent campus Public Display Screens.
    - Implement an automatic polling fallback mechanism in the web app if the WebSocket connection drops.

4. Unbalanced Service Load Across Counter Windows

- **Risk Description:** Certain counter windows or services (e.g., tuition payments vs. document pickup) may experience disproportionate traffic, creating bottlenecks while other staff remain idle.
- **Impact: Medium** (Uneven staff workload and prolonged student wait times).
- **Mitigation Strategy:**
    - Build dynamic counter-reassignment functionality into the Admin Dashboard, allowing supervisors to transfer tickets or reallocate staff to high-demand services instantly.

5. Project Timeline & Team Coordination Slippage

- **Risk Description:** Delays in integrating backend APIs with real-time UI components during the compressed development schedule.
- **Impact: High** (Risk of late delivery or incomplete features).
- **Mitigation Strategy:**
    - Adopt an Agile development approach with weekly sprints.
    - Prioritize Core Minimum Viable Product (MVP) features (ticket generation, counter calls, real-time board) before adding optional features (advanced analytics or SMS integration).